

정말 대단하십니다! 프로그램의 흐름을 직접 테스트해보면서 논리적인 오류(버그)를 정확하게 찾아내셨네요.

결론부터 말씀드리면, 이 현상은 "파이썬의 전역(Global) 변수 지정 선언"을 빼먹었거나, 변수에 대입(=)하는 방식의 차이 때문에 발생한 것입니다. 개발자들이 처음에 정말 많이 겪는 아주 중요하고 좋은 질문입니다.

왜 이런 일이 일어났는지 원인과 해결 방법을 쉽게 설명해 드릴게요.

원인 분석: **cart = []**가 전역 변수를 바꾸지 못한 이유

함수 내부에서 **total_payment = 0**을 했을 때는 상단에 **global total_payment**라고 적어주었기 때문에 바깥에 있는 진짜 장부가 잘 초기화되었습니다.

하지만 **cart = []**를 하실 때 함수 상단에 **global cart**를 적어주지 않았다면, 파이썬은 다음과 같이 오해합니다.

"어? 함수 안에서 cart에 새로운 값을 대입(=)하네? 이건 바깥의 cart가 아니라, **이 함수 안에서만 잠깐 쓰고 버릴 새로운 지역 변수 cart*를 만들라는 뜻이구나!"

즉, 진짜 전역 변수인 **cart** 서랍장은 라면이 들어있는 채로 그대로 남아있고, 결제 함수 안에서만 이름만 똑같은 가짜 빈 서랍장(**cart = []**)을 새로 만들었다가 함수가 끝나면서 사라지게 된 것입니다. 그래서 두 번째 사람이 주문할 때 여전히 라면이 남아있었던 것이죠.

해결 방법 (2가지 중 선택)

방법 ①: 함수 상단에 **global cart** 추가하기 (가장 직관적인 방법)

결제 함수 시작 부분의 **global** 선언에 **cart**를 추가해 주면 **cart = []**가 바깥의 진짜 장바구니를 초기화하게 됩니다.

```
Python
def checkout_and_pay():
    # 🌟 여기에 cart를 꼭 추가해 주어야 합니다!
    global total_payment, today_revenue, my_seat, cart

    # ... (중략) ...

    total_payment = 0
    cart = [] # 이제 진짜 전역 변수 cart가 비워집니다.
```

방법 ②: **cart.clear()** 사용하기 (파이썬에서 추천하는 방법)

global을 추가하기 귀찮다면, **cart = []** 대신 **cart.clear()**를 사용하시면 됩니다.

Python

```
def checkout_and_pay():
```

```
    global total_payment, today_revenue, my_seat # global cart가 없어도 됨
```

```
    # ... (종락) ...
```

```
    total_payment = 0
```

```
    cart.clear() # ★ 전역 변수 지정은 안 해도 완벽하게 비워집니다!
```

💡 왜 **cart.clear()**는 **global** 없이도 작동할까요?

- **cart = []**는 아예 새로운 빈 리스트(새 서랍장)를 변수에 대입하는 행동이라 **global** 선언이 필수입니다.
- 반면 **cart.clear()**는 기존에 쓰던 서랍장은 그대로 둔 채 내용물만 싹 비우는 행동(수정)입니다. 우리가 음식을 담을 때 **cart.append()**를 쓸 때 **global**을 안 적어도 바깥 장바구니에 잘 담겼던 것과 같은 원리입니다.

둘 중 편한 방법을 사용하시면 버그가 완벽하게 해결될 것입니다! 코드가 점점 완성도 높은 실제 프로그램처럼 변해가고 있네요.